BÀI TẬP LÝ THUYẾT
TÌM KIẾM, CSP VÀ GAME TRONG AI

Họ và tên	Lê Minh Hiếu
MSSV / Lớp	3124720023/DNT1241

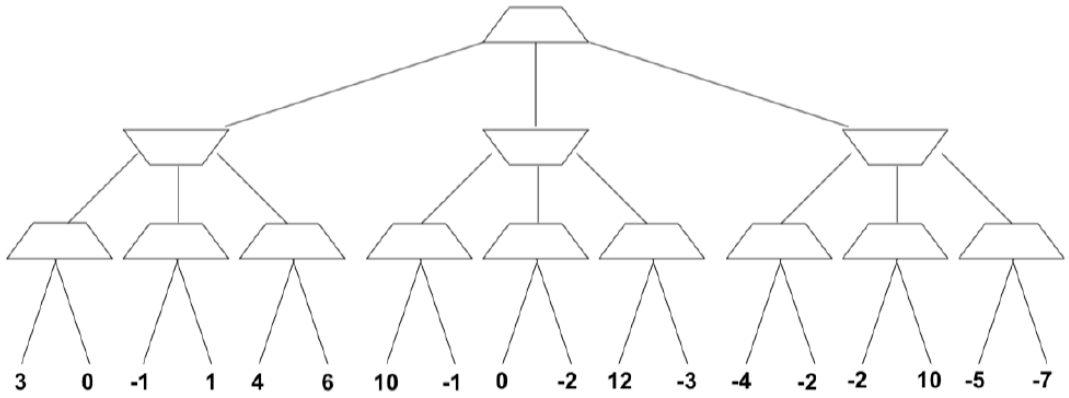
Mục tiêu học tập

- Thực hiện Minimax và Alpha-Beta Pruning, xác định giá trị các nút và các nhánh bị cắt tỉa.
- Phân tích cây trò chơi có nút ngẫu nhiên (chance nodes) và khả năng cắt tỉa.
- Vận dụng Forward Checking, Arc Consistency và suy luận trên đồ thị ràng buộc CSP.
- Giải thích tính đầy đủ, tối ưu của các thuật toán tìm kiếm và tác động khi chi phí cạnh thay đổi.
- Phân tích tác tử, khả năng quan sát của môi trường và tính hợp lý của hành vi.

Hướng dẫn: Trả lời các câu hỏi bằng lập luận ngắn gọn, trình bày các bước tính toán khi cần. Đối với câu có hình, có thể điền trực tiếp vào hình hoặc bổ sung bảng tính.

Bài 1. Minimax và Alpha-Beta Pruning

Xét cây trò chơi tổng bằng không trong Hình 1. Nút gốc là MAX, tầng tiếp theo là MIN và tầng kế tiếp là MAX. Hai người chơi đều hành động tối ưu. Các nút con được xét từ trái sang phải.

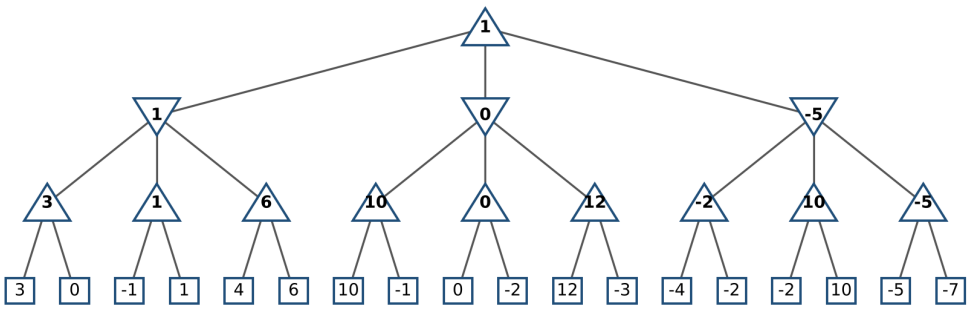


Hình 1. Cây trò chơi dùng cho Minimax và Alpha-Beta (nguồn: rec3.pdf, trang 1)

a) Áp dụng Minimax, tính giá trị mỗi nút trung gian trong cây.

Tính từ dưới lên: nút MAX lấy giá trị lớn nhất của các con, nút MIN lấy giá trị nhỏ nhất. Đặt tên 3 nút MIN là M1, M2, M3 (trái → phải).

- Tầng MAX (dưới cùng): max(3,0) = 3; max(−1,1) = 1; max(4,6) = 6 | max(10,−1) = 10; max(0,−2) = 0; max(12,−3) = 12 | max(−4,−2) = −2; max(−2,10) = 10; max(−5,−7) = −5.
- Tầng MIN: M1 = min(3, 1, 6) = 1; M2 = min(10, 0, 12) = 0; M3 = min(−2, 10, −5) = −5.
- Gốc MAX = max(1, 0, −5) = 1.



Giá trị Minimax tại mọi nút

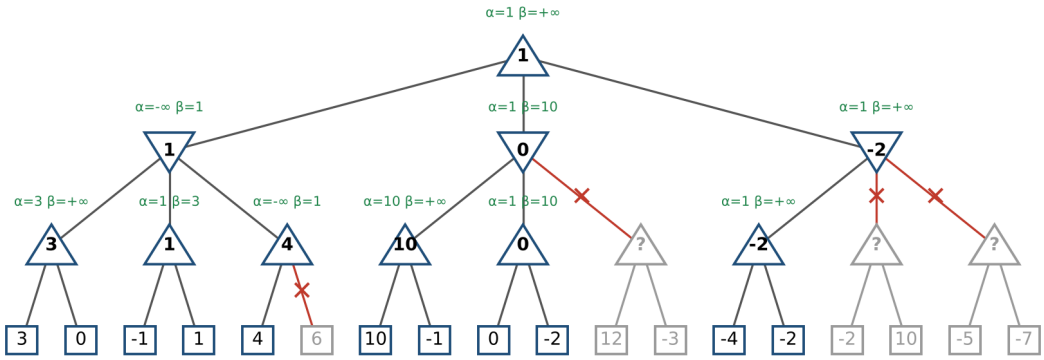
b) MAX nên chọn nhánh nào từ nút gốc? Giá trị trò chơi là bao nhiêu?

MAX nên chọn nhánh trái (vào M1) vì đây là nhánh có giá trị lớn nhất ($1 > 0 > -5$). Giá trị trò chơi là 1: dù MIN chơi tốt đến đâu, đi nhánh trái MAX vẫn chắc chắn có ít nhất 1, còn hai nhánh kia MIN sẽ ép xuống 0 hoặc −5.

c) Áp dụng Alpha-Beta Pruning theo thứ tự trái sang phải. Ghi α, β ở các nút đã thăm; đánh dấu X các nhánh bị cắt tỉa.

α là điểm tốt nhất MAX đã chắc chắn có, β là điểm thấp nhất MIN đã chắc chắn ép được. Khi tại một nút có $v \geq \beta$ (nút MAX) hoặc $v \leq \alpha$ (nút MIN) thì cắt phần còn lại.

- M1 ($\alpha=-\infty, \beta=+\infty$): nút MAX thứ 1 cho 3 → $\beta=3$; nút MAX thứ 2 ($\alpha=-\infty, \beta=3$) cho 1 → $\beta=1$; nút MAX thứ 3 ($\alpha=-\infty, \beta=1$) gặp lá $4 \geq \beta$ → cắt lá 6. M1 = 1, về gốc $\alpha = 1$.
- M2 ($\alpha=1, \beta=+\infty$): nút MAX thứ 4 cho 10 → $\beta=10$; nút MAX thứ 5 ($\alpha=1, \beta=10$) cho 0. Lúc này $M2 \leq 0 \leq \alpha=1$ → cắt nút MAX thứ 6 (lá 12, −3).
- M3 ($\alpha=1, \beta=+\infty$): nút MAX thứ 7 cho $-2 \leq \alpha=1$ → cắt nút MAX thứ 8 và 9 (lá −2, 10, −5, −7).
- Gốc vẫn bằng 1, giống Minimax.



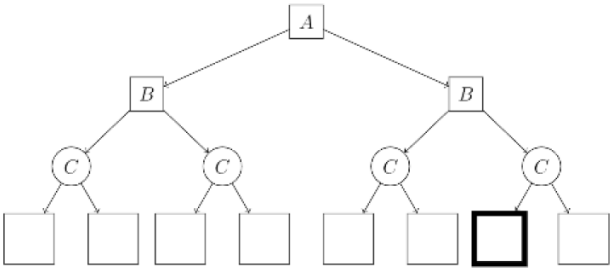
Alpha-Beta: số xanh là (α, β) tại nút, X đỏ là nhánh bị cắt

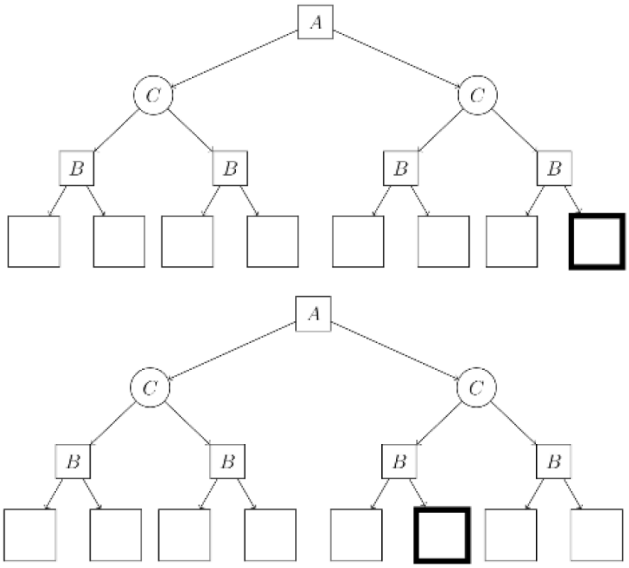
d) Liệt kê các nút lá không cần thăm. So sánh số nút lá được đánh giá bởi Minimax và Alpha-Beta.

Các lá không cần thăm: 6, 12, −3, −2, 10, −5, −7 (7 lá). Minimax phải đánh giá đủ 18 lá, Alpha-Beta chỉ cần 11 lá, tiết kiệm khoảng 39% mà kết quả ở gốc vẫn giữ nguyên.

Bài 2. Alpha-Beta trong cây có nút ngẫu nhiên (Expinimax)

Xét ba cây trò chơi ở Hình 2. A là nút MIN, B là nút MAX và C là nút ngẫu nhiên (chance node). Các con của C có xác suất bằng nhau. Duyệt từ trái sang phải. Với mỗi cây, hãy gán các giá trị hữu hạn cho các nút lá để có thể cắt tỉa nút được viền đậm, hoặc ghi "Không thể" và giải thích. Không thực hiện cắt tỉa trong trường hợp bằng nhau.





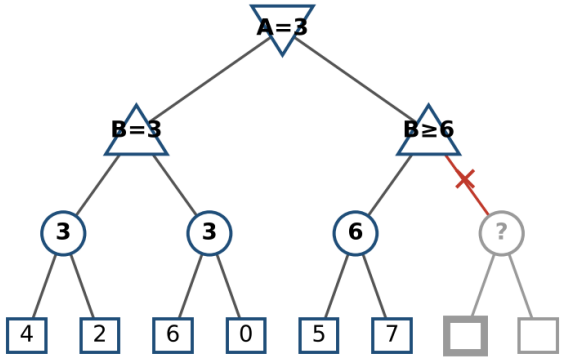
Hình 2. Ba trường hợp cắt tỉa trong cây Expinimax (nguồn: rec3.pdf, trang 3)

a) Trường hợp 1: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể. Gán lá (trái → phải): 4, 2, 6, 0, 5, 7, còn hai lá cuối tùy ý.

- B trái: C = (4+2)/2 = 3 và C = (6+0)/2 = 3 → B trái = 3, nên A (MIN) đã có β = 3.
- B phải: C thứ 3 = (5+7)/2 = 6 → B phải (MAX) ≥ 6 > 3 → A chắc chắn chọn B trái → cắt cả C thứ 4, trong đó có lá viền đậm.

Điều kiện cắt: giá trị C thứ 3 > giá trị B trái (lớn hơn hẳn, không bằng).

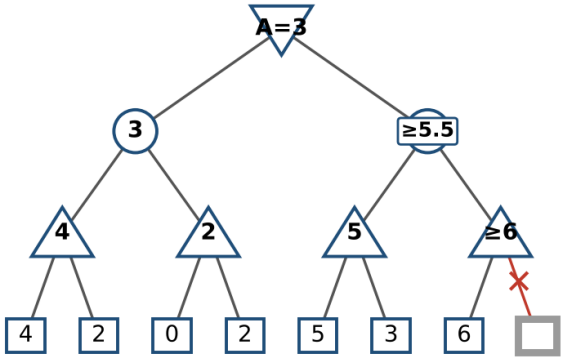


b) Trường hợp 2: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Có thể. Gán lá: 4, 2, 0, 2, 5, 3, 6, lá viền đậm bị cắt.

- C trái = (max(4,2) + max(0,2))/2 = (4+2)/2 = 3 → β của A là 3.
- C phải: B thứ 3 = max(5,3) = 5; B thứ 4 gặp lá 6 nên B thứ 4 ≥ 6 → C phải ≥ (5+6)/2 = 5,5 > 3 → A không chọn C phải → cắt lá viền đậm.

Điều kiện cắt: (B thứ 3 + lá thứ 7)/2 > C trái. Cắt được là nhờ nút B là MAX nên có ngay cận dưới sau khi thấy một lá.



c) Trường hợp 3: điền các giá trị lá (nếu có thể) và chỉ ra điều kiện cắt tỉa.

Không thể. Khi đến lá viền đậm, ta mới biết C trái và lá đầu của B thứ 3, còn B thứ 4 chưa xem lá nào. Muốn A bỏ C phải thì cần C phải = $(B_3 + B_4)/2 >$ C trái, nhưng B_4 có thể nhỏ tùy ý nên không có cận dưới. Cắt ngay trong B_3 cũng không được, vì cha của B_3 là nút ngẫu nhiên, phụ thuộc B_4 chưa biết, nên không truyền xuống được cận trên hữu hạn. Vì vậy với giá trị lá không bị chặn thì gán gì cũng không cắt được (chỉ khi biết trước lá nằm trong một khoảng, ví dụ $[-10, 10]$, mới có thể cắt).

d) Giải thích vì sao cắt tỉa trong cây có nút ngẫu nhiên khác với Alpha-Beta trong cây Minimax.

Ở cây Minimax, giá trị nút MIN/MAX bằng đúng một đứa con, nên chỉ cần thấy một con là có ngay cận (MAX ≥ con đó, MIN ≤ con đó) để cập nhật α, β và cắt. Còn nút ngẫu nhiên lấy trung bình tất cả các con; chỉ một đứa con chưa xem cũng có thể kéo trung bình đi xa tùy ý, nên nếu giá trị lá không bị chặn thì nút ngẫu nhiên không cho ra cận nào. Vì vậy trong Expectimax chỉ cắt được khi cận đến từ nút MIN/MAX (như trường hợp 1, 2) hoặc khi biết trước miền giá trị lá bị chặn, nên cắt tỉa hạn chế hơn nhiều.

Bài 3. Câu hỏi đúng/sai

Xét cây trò chơi có gốc MAX, giá trị Minimax là V_M. Thay mọi nút MIN bằng nút ngẫu nhiên có phân phối xác suất bất kỳ nhưng đã biết, ta nhận cây có giá trị Expectimax là V_E. Với mỗi mệnh đề, chọn Đúng/Sai và giải thích.

a) Luôn có V_M ≤ V_E. ☑ **Đúng** ☐ Sai

Trung bình có trọng số của các con không bao giờ nhỏ hơn con nhỏ nhất, nên thay nút MIN bằng nút ngẫu nhiên thì giá trị nút đó chỉ tăng hoặc giữ nguyên. Nút MAX lấy max nên giữ nguyên chiều so sánh đó, đi ngược lên tới gốc ta được V_M ≤ V_E. Nói gọn: đối thủ chơi ngẫu nhiên không thể tệ cho MAX hơn đối thủ chơi ác nhất.

b) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_M. ☑ **Đúng** ☐ Sai

Chính sách Minimax đảm bảo MAX được ít nhất V_M với **bất kỳ** cách chơi nào của đối thủ, kể cả cách tệ nhất. Đối thủ ngẫu nhiên chỉ là một trường hợp trong số đó, nên kỳ vọng vẫn ≥ V_M.

c) Dùng chính sách Minimax tối ưu ở cây đã thay nút MIN bằng nút ngẫu nhiên sẽ đảm bảo lợi ích kỳ vọng ít nhất là V_E. ☐ Đúng ☑ **Sai**

Chính sách Minimax quá phòng thủ nên có thể bỏ qua nước có kỳ vọng cao. Phản ví dụ: gốc MAX có hai nhánh, nhánh trái là nút ngẫu nhiên với lá {0, 100}, nhánh phải là nút ngẫu nhiên với lá {1,

1}. Minimax coi trái = 0, phải = 1 nên chọn phải và chỉ được kỳ vọng 1. Trong khi đó V_E = max(50, 1) = 50 > 1.

Xét trò chơi một người, kết quả hành động có thể không xác định trước. Người chơi nhận +1 khi thắng và −1 khi thua. Đánh dấu mọi phương pháp có thể phù hợp để mô hình hóa và giải trò chơi theo giả định cần thiết.

☑ Minimax với tiện ích trạng thái cuối +1/−1

☑ Expectimax với tiện ích trạng thái cuối +1/−1

☑ Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua

☐ Không phương án nào

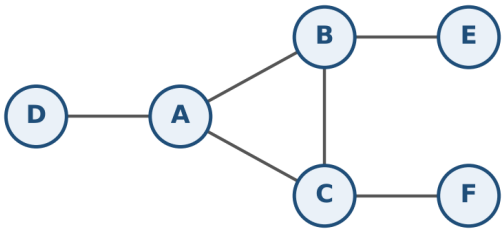
d) Giải thích ngắn gọn lựa chọn ở trên và những giả định cần thiết.

- Expectimax: phù hợp khi biết xác suất kết quả của mỗi hành động và cây trò chơi hữu hạn; các nút ngẫu nhiên đóng vai "tự nhiên" và lấy kỳ vọng.
- Value Iteration: coi trò chơi là một MDP với mô hình chuyển trạng thái đã biết; ưu điểm là xử lý được cả khi trạng thái lặp lại. Với $\gamma = 1$, giá trị chính là xác suất thắng trừ xác suất thua.
- Minimax: dùng được nếu giả định "tự nhiên" luôn chọn kết quả xấu nhất cho người chơi, tức là muốn tối đa hóa trường hợp xấu nhất. Kết quả là một mức đảm bảo an toàn nhưng thường bi quan hơn thực tế.

Bài 4. CSP với Constraint Graph và Lan truyền ràng buộc

Có 6 sinh viên A, B, C, D, E, F cần sắp chỗ ngồi. Các cặp có ràng buộc nhị phân: (A,B), (A,C), (A,D), (B,C), (B,E), (C,F).

a) Vẽ đồ thị ràng buộc tương ứng (mỗi biến là một đỉnh; mỗi ràng buộc nhị phân là một cạnh).



Đồ thị có 6 đỉnh, 6 cạnh. A, B, C tạo thành tam giác; D chỉ nối A, E chỉ nối B, F chỉ nối C.

b) Sau khi gán giá trị cho A, miền của những biến nào có thể bị thay đổi bởi Forward Checking?

Forward Checking chỉ xét các hàng xóm trực tiếp chưa gán của A, nên miền có thể đổi là B, C, D.

c) Sau khi gán A và thực hiện Forward Checking, tiếp tục gán B: miền của những biến nào có thể bị thay đổi khi Forward Checking tại B?

Hàng xóm của B là A, C, E; A đã gán rồi nên chỉ còn C và E.

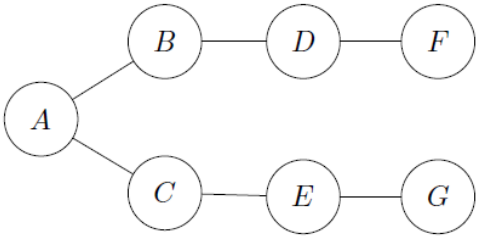
d) Sau khi gán A, nếu duy trì Arc Consistency, miền của những biến nào có thể thay đổi?

Arc Consistency lan truyền tiếp: B, C, D bị thu hẹp trực tiếp; khi B, C đổi thì E (nối B) và F (nối C) cũng có thể bị ảnh hưởng. Vậy là B, C, D, E, F.

e) Sau khi gán A và duy trì Arc Consistency, tiếp tục gán B và duy trì Arc Consistency: miền của những biến nào có thể thay đổi?

Từ B lan sang C và E, từ C lan tiếp sang F. Riêng D chỉ nối với A, mà A đã gán và đã xử lý ở bước trước, nên không có thay đổi mới truyền tới D. Đáp án: C, E, F.

Xét thuật toán mới: thực hiện Arc Consistency ban đầu và sau mỗi lần đã gán một số chẵn biến (2, 4, 6, ...). Mỗi biến có miền kích thước d. Sử dụng đồ thị ở Hình 3 để phân tích số lần backtrack trong trường hợp xấu nhất cho từng thứ tự gán; nêu rõ cách tính và các giả định.



Hình 3. Đồ thị ràng buộc cho bài toán thứ tự gán biến (nguồn: rec3.pdf, trang 6)

Giả định chung: đồ thị Hình 3 thực chất là một đường thẳng F – D – B – A – C – E – G (dạng cây). Trường hợp xấu nhất, AC ban đầu không loại được giá trị nào. Giá trị mâu thuẫn trực tiếp với hàng xóm đã gán thì bỏ qua luôn, không tính. Một lần backtrack là gán một giá trị, chạy AC thấy miền rỗng, rồi phải gỡ giá trị đó ra để thử giá trị khác. Ta dùng tính chất của CSP dạng cây: khi cả cây đã arc consistent thì mọi giá trị còn trong miền đều mở rộng được thành lời giải, và nếu mỗi biến khi được gán chỉ nối với đúng một biến đã gán thì luôn tìm được giá trị hợp lệ.

f) Thứ tự A → B → C → D → E → F → G

Mỗi biến khi được gán chỉ có đúng một hàng xóm đã gán trước (B, C nối A; D nối B; E nối C; F nối D; G nối E). Giá trị của biến cha được lấy từ miền đã arc consistent nên luôn có giá trị phù hợp trong miền của con, kể cả khi AC chưa chạy ngay sau biến cha. Các cặp gán liền nhau mà không nối nhau (như C rồi D) chỉ liên hệ qua những biến đã gán. → 0 lần backtrack.

g) Thứ tự G → E → C → A → B → D → F

Đây là đi dọc đường thẳng từ đầu G sang đầu F: mỗi biến mới chỉ kề đúng biến vừa gán trước nó. Lập luận giống câu f → 0 lần backtrack.

h) Thứ tự G → F → E → D → C → B → A

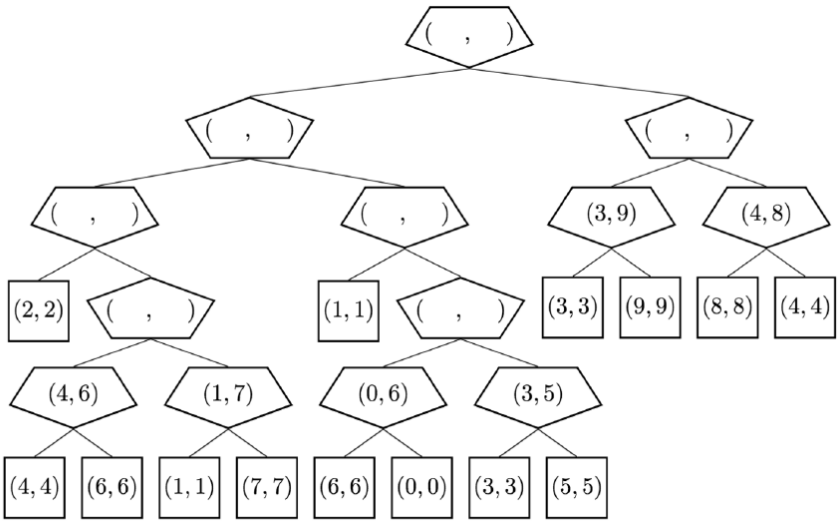
G và F nằm ở hai đầu đường thẳng, gán liên tiếp mà không có AC xen giữa, nên cặp (G, F) có thể không cùng nằm trong lời giải nào:

- Gán F (bước 2): AC sau đó có thể làm rỗng miền, tệ nhất thử sai d − 1 giá trị. Chắc chắn có giá trị đúng vì G lấy từ miền đã AC.
- Gán E rồi D (bước 3–4): E kề G nên ổn, nhưng E và D liên hệ qua C – A – B chưa gán → tệ nhất d − 1 lần ở D.
- Gán C rồi B (bước 5–6): C và B cùng nối với A chưa gán → tệ nhất d − 1 lần ở B.
- Gán A (bước 7): sau AC ở bước 6, miền A chắc chắn còn giá trị hợp lệ → 0 lần.

→ Tổng tệ nhất 3(d − 1) lần backtrack, và không bao giờ phải lùi về biến trước đó. Thứ tự này kém vì hai biến gán liên tiếp không được AC kiểm tra chéo với nhau.

Bài 5. Cây trò chơi Pacman và Casper

Pacman tìm cách tối đa hóa tiện ích. Casper là một bóng ma thân thiện, thực sự muốn tăng tiện ích cho Pacman nhưng Pacman tưởng Casper cố giảm tiện ích. Casper biết Pacman đang hiểu nhầm. Ở mỗi nút, cặp (u, v) biểu thị lần lượt giá trị mà Pacman tin tưởng và giá trị theo hành vi thực tế của Casper. Tại lá, u = v = tiện ích cuối.

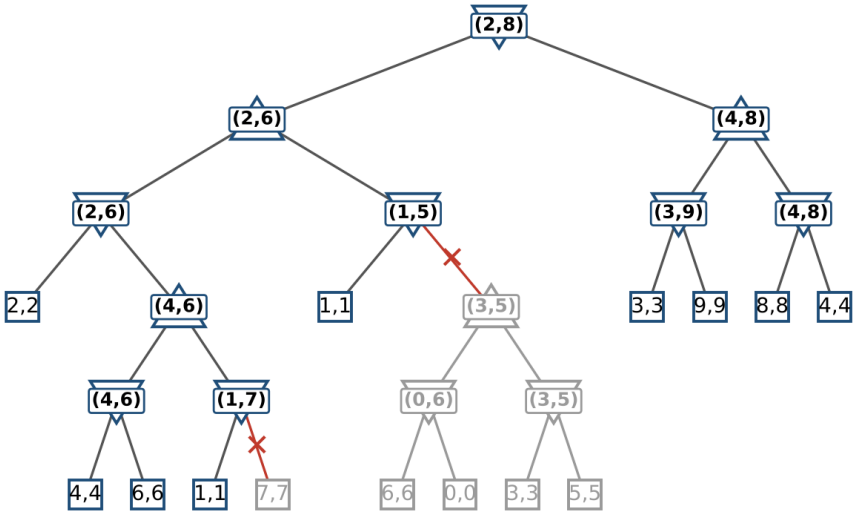


Hình 4. Cây trò chơi Pacman–Casper

a) Điền tất cả cặp (u, v) còn thiếu trong Hình 4. Giải thích sự khác biệt giữa u và v tại nút Casper.

Các tầng xen kẽ: gốc là Casper, tầng 1 là Pacman, tầng 2 là Casper (như (3,9)), tầng 3 là Pacman, tầng 4 là Casper (như (4,6)). Quy tắc: ở nút Casper, u = min các u con, v = max các v con. Ở nút Pacman, Pacman chọn con có u lớn nhất, nên u = max các u con, còn v = v của chính đứa con được chọn.

Nút còn thiếu	Loại	Kết quả
Cha của (4,6) và (1,7)	Pacman	chọn (4,6) vì 4 > 1 → (4, 6)
Cha của (0,6) và (3,5)	Pacman	chọn (3,5) vì 3 > 0 → (3, 5)
Cha của (2,2) và (4,6)	Casper	u = min(2,4) = 2, v = max(2,6) = 6 → (2, 6)
Cha của (1,1) và (3,5)	Casper	u = min(1,3) = 1, v = max(1,5) = 5 → (1, 5)
Nút trái tầng 1	Pacman	chọn (2,6) vì 2 > 1 → (2, 6)
Nút phải tầng 1	Pacman	chọn (4,8) vì 4 > 3 → (4, 8)
Gốc	Casper	u = min(2,4) = 2, v = max(6,8) = 8 → (2, 8)



Hình 4 đã điền (tam giác lên: Pacman, tam giác xuống: Casper)

Khác biệt giữa u và v tại nút Casper: u là giá trị trong suy nghĩ của Pacman. Pacman tưởng Casper là ma xấu nên sẽ chọn nước tệ nhất cho mình (min). v là điều thật sự xảy ra: Casper tốt bụng nên chọn nước tốt nhất cho Pacman (max), và vì biết Pacman hiểu nhầm nên Casper đoán trước được Pacman sẽ đi đâu. Kết quả là Pacman nghĩ ván này chỉ được 2 điểm, nhưng thực tế được 8.

b) Theo thứ tự duyệt trái sang phải, đánh dấu các nhánh có thể không cần duyệt khi Casper tính giá trị trò chơi.

Ở nút Casper cần max của mọi v nên phải xét hết. Có thể cắt ở nút Pacman: Pacman chọn theo u, mà u của nút Casper con là min, nên có cận trên ngay khi thấy một lá; nếu cận đó đã thấp hơn lựa chọn Pacman đang có thì Pacman sẽ không chọn nhánh đó.

- Lá (7, 7) dưới nút (1, 7): Pacman đã có u = 4, thấy lá 1 thì $u \leq 1 < 4 \rightarrow$ bỏ qua.
- Cả cây con dưới nút Pacman (3, 5), gồm (0,6), (3,5) và 4 lá (6,6), (0,0), (3,3), (5,5): Pacman ở tầng 1 đã có u = 2, nút (1, 5) có lá đầu là 1 nên $u \leq 1 < 2 \rightarrow$ không cần duyệt tiếp.
- Nhánh phải ((3,9), (4,8)) phải duyệt hết vì cần u chính xác để so 3 với 4, và cần v của nhánh được chọn.

c) Nếu Pacman biết Casper thân thiện thì giá trị toàn bộ cây thay đổi như thế nào?

Khi đó cả hai cùng tối đa hóa nên mọi nút đều là MAX và u = v ở mọi nút. Giá trị cây là lá lớn nhất: nhánh phải → (3,9) → lá 9 (nhánh trái cao nhất chỉ đến 7). Giá trị trò chơi tăng từ 8 lên 9 vì Pacman không còn sợ Casper nên dám vào nhánh (3, 9) thay vì (4, 8).

Bài 6. Thuật toán tìm kiếm – Tính đầy đủ và tối ưu

Với mỗi thuật toán sau, xác định điều kiện để được đảm bảo đầy đủ (Complete) và tối ưu (Optimal). Phân biệt tìm kiếm trên cây và trên đồ thị nếu cần.

Thuật toán	Điều kiện đầy đủ	Điều kiện tối ưu
Breadth-First Search (BFS)	Hệ số phân nhánh b hữu hạn (lời giải ở độ sâu hữu hạn). Đúng cho cả cây và đồ thị.	Mọi bước có chi phí bằng nhau (hoặc chi phí không giảm theo độ sâu).
Depth-First Search (DFS)	Trên cây: không đầy đủ (đi sâu vô hạn, lặp vòng). Trên đồ thị: đầy đủ nếu không gian trạng thái hữu hạn.	Không tối ưu (trả về lời giải đầu tiên gặp).
Iterative Deepening Search (IDS)	b hữu hạn (giống BFS).	Chi phí mỗi bước bằng nhau (giống BFS).
A* Search	b hữu hạn và mỗi bước có chi phí $\geq \varepsilon > 0$.	Trên cây: heuristic chấp nhận được (admissible). Trên đồ thị: heuristic nhất quán (consistent).

Giải thích vì sao điều kiện chi phí bước và đặc tính heuristic có thể ảnh hưởng đến kết luận.

- Chi phí bước: BFS và IDS tìm lời giải ít bước nhất chứ không phải rẻ nhất. Nếu chi phí các bước khác nhau, đường ít bước có thể đắt hơn → mất tối ưu (cần UCS hoặc A*). Với A*, nếu chi phí có thể nhỏ dần về 0 thì thuật toán có thể đi mãi trên một chuỗi vô hạn có tổng hữu hạn → mất đầy đủ, nên cần chi phí $\geq \varepsilon$.
- Heuristic: admissible (không ước lượng quá) giúp A* không bỏ nhầm đường tối ưu vì tưởng nó đắt. Nhưng trên đồ thị, mỗi trạng thái chỉ được mở rộng một lần; nếu h chỉ admissible mà không consistent, lần đầu tới một trạng thái có thể chưa phải đường rẻ nhất mà trạng thái đã bị đóng

→ mất tối ưu. Consistent đảm bảo f không giảm dọc đường đi, nên lần đầu lấy trạng thái ra chính là đường rẻ nhất tới nó.

Bài 7. A* khi chi phí một cạnh thay đổi

Sau khi A* graph search tìm được đường đi tối ưu từ trạng thái đầu đến đích, chi phí cạnh X → Y thay đổi từ c thành c + n hoặc c − n, với n > 0. Với từng tình huống, hãy phân tích liệu chi phí tối ưu có thay đổi và cách cập nhật/tìm đường tối ưu mới thay vì nhất thiết chạy lại từ đầu. Khi giảm chi phí, giả sử chi phí mới vẫn hợp lệ theo mô hình tìm kiếm.

Gọi C* là chi phí tối ưu cũ; giả sử heuristic vẫn admissible/consistent với chi phí mới.

a) Chi phí tăng n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Có thể thay đổi. Đường cũ giờ tốn C* + n, có thể có đường khác (không qua X → Y) rẻ hơn mức đó; chi phí tối ưu mới nằm trong [C*, C* + n]. **Cập nhật:** cộng n vào g của Y và các nút con cháu của Y trong cây tìm kiếm, đưa các nút này (kể cả đích) trở lại hàng đợi với f mới rồi **tiếp tục A*** từ hàng đợi hiện có cho đến khi lấy được đích.

b) Chi phí giảm n; cạnh X → Y thuộc đường tối ưu cũ và đã được khám phá.

Đường tối ưu giữ nguyên, chi phí giảm còn C* − n. Các đường không qua X → Y vẫn tốn ≥ C*; các đường qua X → Y đều giảm cùng một lượng n, mà đường cũ vốn rẻ nhất trong nhóm này. → Không cần tìm lại, chỉ cập nhật chi phí.

c) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Không thay đổi. Đường tối ưu cũ không đi qua cạnh này nên vẫn tốn C*, các đường khác chỉ có thể đắt lên. Giữ nguyên kết quả.

d) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ nhưng đã được khám phá.

Có thể thay đổi, nếu g(X) + c − n + (chi phí tốt nhất từ Y đến đích) < C*. **Cập nhật:** đặt g(Y) = min(g(Y), g(X) + c − n); nếu g(Y) giảm thì cập nhật các con cháu của Y, mở lại Y (đưa vào hàng đợi) và **tiếp tục A*** cho tới khi lấy được đích. Phần còn lại của cây tìm kiếm vẫn dùng được.

e) Chi phí tăng n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Không thay đổi. A* chưa cần đụng tới cạnh này đã tìm được đường tối ưu, cạnh đắt hơn thì càng vô dụng. Không cần làm gì.

f) Chi phí giảm n; cạnh X → Y không thuộc đường tối ưu cũ và chưa được khám phá.

Không thay đổi. Cạnh chưa được khám phá nghĩa là X chưa được mở rộng, tức f(X) = g(X) + h(X) ≥ C* (A* lấy nút theo f tăng dần và đích đã ra trước X). Mọi đường qua X tốn ít nhất g(X) + (chi phí thật từ X đến đích) ≥ g(X) + h(X) ≥ C*, nên dù cạnh rẻ hơn thì đường qua X vẫn không rẻ hơn C*. Kết quả cũ vẫn tối ưu.

Bài 8. Tác tử thông minh và môi trường

Xét ba kiểu tác tử, trong đó các hàm f_A, f_B, f_C có thể ngẫu nhiên nhưng không lưu trạng thái nội bộ:

- A: act(percept) → f_A()
- B: act(percept) → f_B(percept)
- C: lưu danh sách percepts; mỗi lần act(percept) thêm percept rồi trả về f_C(percepts)

Với mỗi đặc tính môi trường sau, hãy liệt kê các lớp tác tử có thể tồn tại một hàm f để hành động hợp lý hoàn hảo trong mọi môi trường thuộc lớp đó. Giải thích ngắn gọn.

a) Quan sát đầy đủ; xác định; rời rạc; một tác tử; tĩnh.

B và C. Quan sát đầy đủ nên percept cho biết trạng thái thật; môi trường xác định nên hành động tốt nhất chỉ phụ thuộc trạng thái hiện tại → B chỉ cần ánh xạ trạng thái → hành động tốt nhất. C có thể làm như B (chỉ dùng percept cuối). A không nhìn percept nên thua ngay khi hai trạng thái khác nhau cần hai hành động khác nhau.

b) Quan sát một phần; xác định; rời rạc; một tác tử; tĩnh.

Chỉ C. Một percept có thể ứng với nhiều trạng thái khác nhau nên B không phân biệt được (ví dụ robot hút bụi chỉ thấy ô hiện tại, không biết ô kia đã dọn chưa, nên có thể đi vòng vòng mãi). C nhớ lịch sử, và vì môi trường xác định nên từ lịch sử suy ra được trạng thái để hành động đúng. B có ngẫu nhiên thì thoát được vòng lặp nhưng không hợp lý hoàn hảo.

c) Quan sát một phần; ngẫu nhiên; rời rạc; một tác tử; tĩnh.

Chỉ C. Hành động hợp lý phụ thuộc belief state (phân phối xác suất về trạng thái hiện tại), mà belief state được tính từ toàn bộ lịch sử percept và hành động. Chọn f_C tất định thì các hành động cũ cũng suy lại được từ lịch sử, nên C tính được belief state và chọn hành động tối đa hóa kỳ vọng. A và B thiếu thông tin.

d) Đúng hay sai: Có tồn tại một môi trường mà mọi tác tử đều hành động hợp lý. Nêu ví dụ hoặc phản biện.

Đúng. Ví dụ một môi trường chỉ có một hành động duy nhất, hoặc mọi hành động đều cho cùng một điểm hiệu năng (hàm đánh giá luôn trả về 0). Khi đó chọn gì cũng là tốt nhất, nên mọi tác tử đều hợp lý.

— Hết —